

Design Review & Risk-Based Test Planning

Semakan Reka Bentuk & Perancangan Ujian Berasaskan Risiko

Evaluate the system design before development, and plan testing where risk is highest, practised end-to-end on one project: ShopFast checkout.

DURATION

90 min

FORMAT

~80% hands-on

PROJECT

ShopFast

KRISA 2.0

Ch.4 · SDS

Jadual slot: Modul 3: Semakan Reka Bentuk dan Ujian Berasaskan Risiko

Run sheet / session timing

Hari 1, 14:00 hingga 15:30 · 90 minit · kira-kira 80% amali



Masa	Segmen	Tempoh
14:00 hingga 14:25	Konsep	25 minit
14:25 hingga 15:05	Makmal	40 minit
15:05 hingga 15:25	Bengkel AI W03	20 minit
15:25 hingga 15:30	Rumusan	5 minit

KRISAv2: 6 fasa, 18 templat serahan, satu aliran DevOps

KRISAv2: 6 phases, 18 deliverable templates, one DevOps flow

Setiap fasa melalui Gate Review dengan senarai semak serahan dan Definition of Done (Bab 8, 8.3.1)

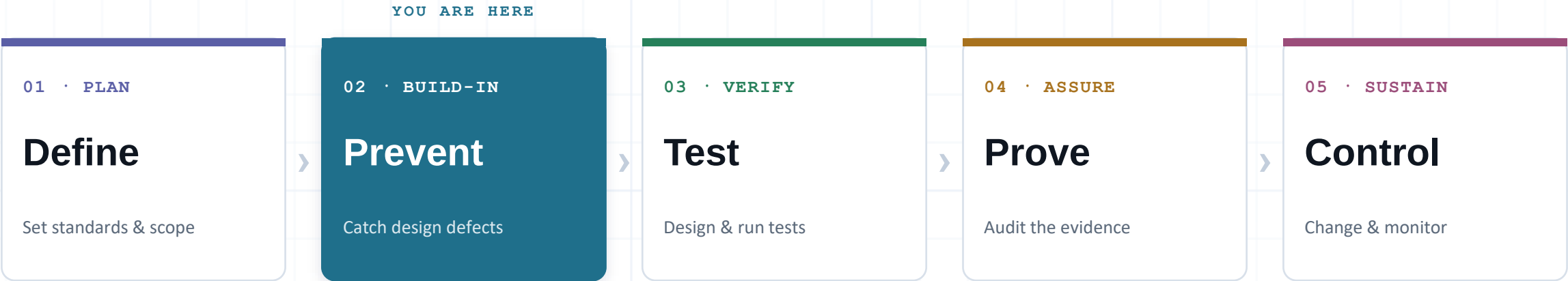
1Permulaan <i>Initiation</i> PPP / PPrISA02 D02 BRS RTM awal (T2.8) KRISAv2 Bab 2M1, M2	2Analisis <i>Analysis</i> D03 SRS D05 Pelan Migrasi D07 Pelan Integrasi KRISAv2 Bab 3M2	3Reka Bentuk <i>Design</i> D04 SDS D06 Spes. Migrasi D08 Spes. Integrasi KRISAv2 Bab 4M3	4Pembangunan <i>Development</i> D09 Pangkalan Data D10 Kod Sumber D11 Laporan Ujian Sistem KRISAv2 Bab 5M4, M5, M6	5Pengujian Penerimaan <i>Acceptance Testing</i> D12 Pelan Induk Pengujian D13 Pelan UAT/PAT D14 Laporan UAT/PAT KRISAv2 Bab 6M4, M6, M7	6Pelaksanaan <i>Implementation</i> D15 Laporan Migrasi D16 Laporan Penamatan Ujian D17 Manual, D18 Serahan KRISAv2 Bab 7M7, M8
--	--	---	---	--	---

Bab 8 Pengelaran DevOps	Perancangan	Pra-Release dan Release: Analisis + Reka Bentuk + Pembangunan + Pengujian dalam sprint, CI/CD, Laporan Release	Pengoperasian: APM, log pemantauan
-------------------------	-------------	--	------------------------------------

PPrISA 2.0	Tadbir urus projek: DPP PPrISA01, PPP PPrISA02, Kualiti PPrISA05, Risiko PPrISA07, Pindaan PPrISA10/11, Status PPrISA16/17, Penamatan PPrISA18
------------	--

Where this module sits in the quality flow

Nine SQA activities map onto five phases. You move one feature, ShopFast, left to right. Module 3 lives in Build-in.



The cheapest defect is one you never build. Design review catches problems while they are still on paper, before code, before test data, before production. That is the whole point of the Build-in phase.

What you will be able to do

By the end of this 90-minute module, on the ShopFast checkout feature.



Read a system design

01

Interpret an architecture diagram, API contract and data model, and judge how testable each part is.



Run a structured design review

02

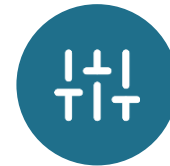
Walk a review checklist to surface defects before a single line of code is written.



Assess design-time security

03

Evaluate the design against [OWASP Top 10](#) (2025) and lightweight threat modelling.



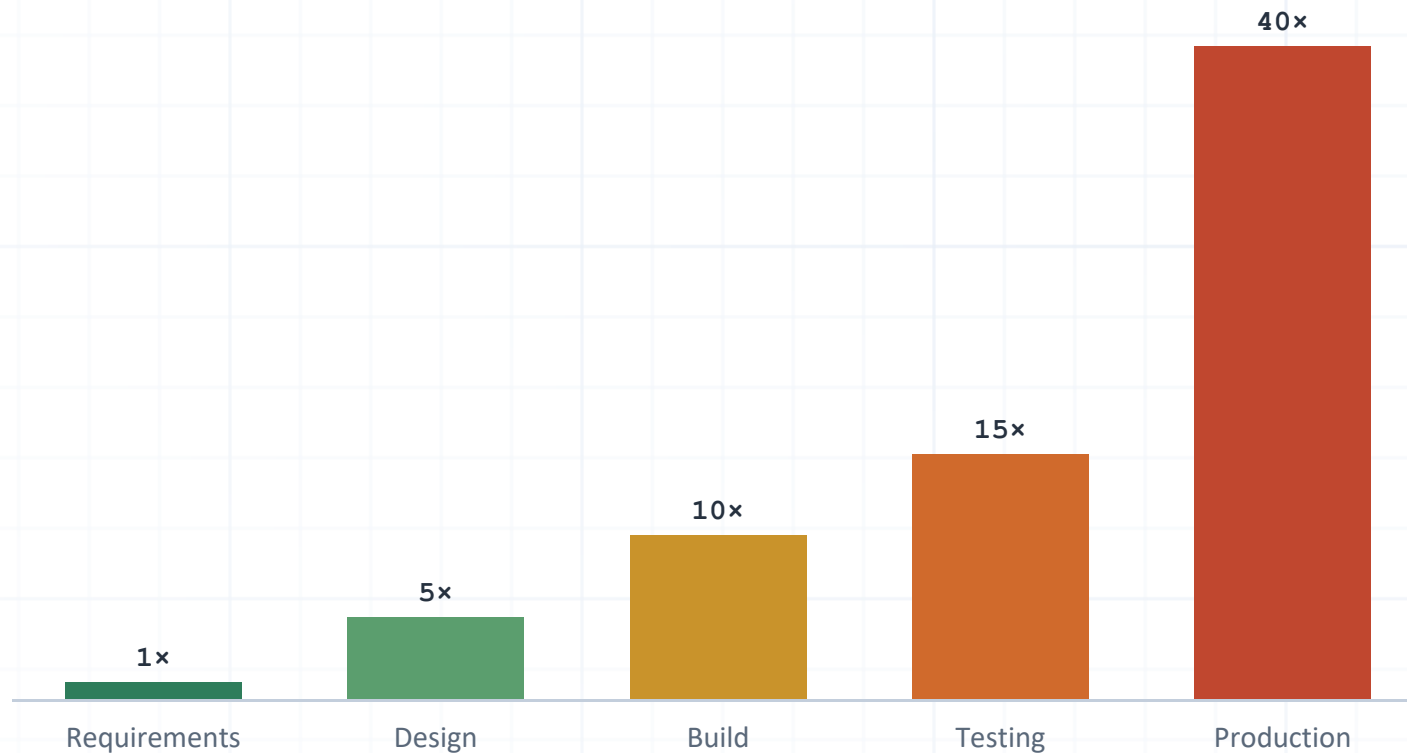
Prioritise testing by risk

04

Score risks by likelihood × impact and turn them into a focused, risk-based test strategy.

The cost of a defect grows with every phase

Fix it in design and it is a conversation. Fix it in production and it is an incident. This is why we shift left.



Relative cost to fix the same defect, by the phase it is caught · illustrative, exact multipliers vary by study

THE ECONOMICS

Design review is prevention, not inspection.

- ✓ A defect caught in design costs a rewrite of a sentence.
- ✓ The same defect in production costs hotfixes, rollbacks and trust.
- ✓ Requirement & design reviews are the highest-leverage QA activities you have.

Two disciplines, one continuous move

A design review surfaces what could go wrong; risk ranking decides where the testing goes.



A · Design Review

Static · preventive

A structured inspection of the design itself. No code runs, you examine architecture, interfaces, data and security decisions while they are still cheap to change.

PRODUCES Findings log + a completed Design Review Checklist



B · Risk-Based Test Planning

Focused · prioritised

You cannot test everything equally. Rank what could go wrong by likelihood × impact, then aim the deepest testing at the areas where failure hurts most.

PRODUCES A scored Risk Register + a Risk-Based Test Strategy

Review surfaces the risks to the risk ranking directs the testing

What a design review inspects

Four lenses. Each asks: is this correct, testable and safe, before we build it?



Architecture

Boundaries & coupling · single points of failure · scalability · failure & recovery modes.



API contract

Endpoints · status codes · request/response schemas · error model · idempotency · versioning.



Data model

Entities & keys · referential integrity · PII handling · migration & retention.

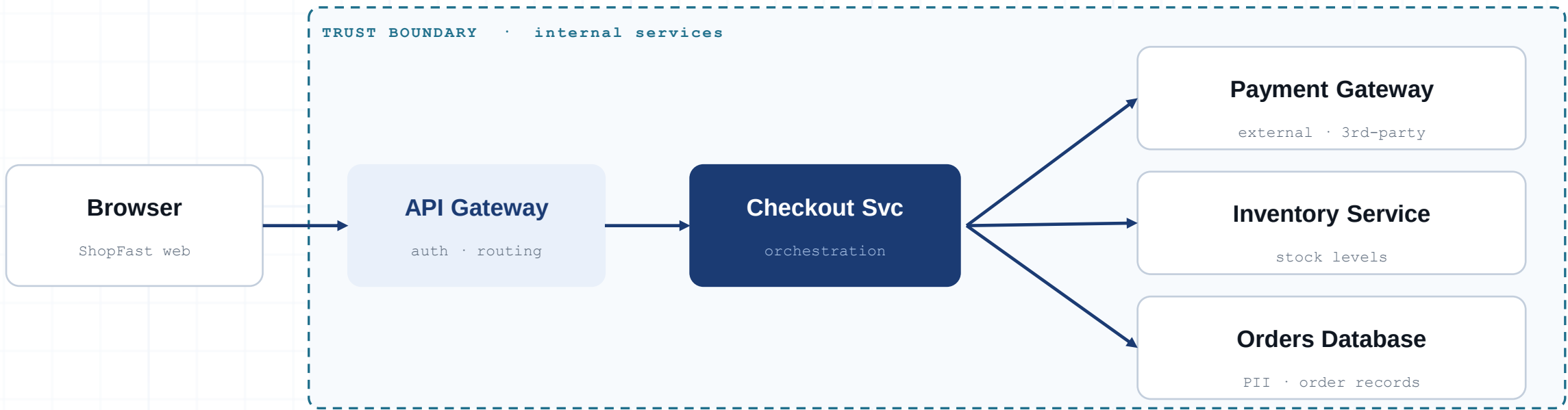


Security

Access control · secrets & input handling · OWASP-aligned design flaws.

The design we review: ShopFast checkout

Trace one checkout request end-to-end. At every hop, ask what could break, and how you would test it.



REVIEW FOCUS AT EACH HOP

- Auth & authz at the gateway
- Idempotent checkout (no double-charge)
- Timeout + retry to payment
- Stock race on concurrent buys
- PII protection in the Orders DB

Review the contract, not just the happy path

An [OpenAPI](#) / [Swagger](#) spec is a testable promise. Read it like a tester: where are the failure paths?

```
checkout.openapi.yaml
```

```
POST /api/checkout
```

```
  Idempotency-Key: <uuid>
```

```
  body: { cartId, payment,  
         address }
```

```
responses:
```

```
  200 order confirmed
```

```
  400 validation error
```

```
  409 stock conflict
```

```
  422 payment declined
```

```
  5xx upstream failure
```

READ THE SPEC FOR

- ✓ Every response documented, 2xx, 4xx and 5xx, not only success
- ✓ A consistent error schema: code · message · field
- ✓ Idempotency on checkout / payment to prevent double-charge
- ✓ Explicit authentication & authorization per endpoint
- ✓ Typed input constraints you can assert against
- ✓ A versioning strategy for breaking changes

Review the design against OWASP Top 10 : 2025

The current edition, released November 2025, the first update since 2021. Four categories are decided at design time.

A01	Broken Access Control	A06	Insecure Design
A02	Security Misconfiguration	A07	Authentication Failures
A03	Software Supply Chain Failures	A08	Software / Data Integrity Failures
A04	Cryptographic Failures	A09	Security Logging & Alerting Failures
A05	Injection	A10	Mishandling of Exceptional Conditions



Fix at design, not in a pen-test. A01 access control, A02 misconfiguration, A06 insecure design and A10 mishandling of exceptional conditions are architecture choices, a review catches them; a scanner mostly cannot.

Source: [OWASP Top 10](#) : 2025, [owasp.org/Top10](#) (highlighted = predominantly design-time).

Threat-model the design with STRIDE

A structured way to ask "what can go wrong?", one threat class at a time, mapped onto ShopFast.

THREAT	SHOPFAST EXAMPLE	DESIGN MITIGATION
S Spoofing	Someone poses as another shopper at checkout	Strong auth · session integrity
T Tampering	Cart total or price altered in transit	Server-side validation · signing
R Repudiation	User denies placing a paid order	Immutable audit logs
I Info disclosure	Card data or PII leaks from Orders DB	Encryption · data minimisation
D Denial of service	Checkout flooded, real buyers blocked	Rate limiting · quotas
E Elevation of priv.	A shopper reaches admin actions	Least privilege · authz checks

The four questions: What are we building? · What can go wrong? · What will we do about it? · Did we do a good job?

The Design Review Checklist

Three lenses you walk on every review. Each unchecked box is a finding, and a candidate test case.



Testability

- ☒ Observable output for each function
- ☒ Test data & environments definable
- ☒ Interfaces mockable / stubbable
- ☒ States you can set up and reset



Error handling

- ☒ Every failure path is defined
- ☒ Fails closed, returns a clear error
- ☒ Timeouts, retries & rollback set
- ☒ No silent or undefined states



Security (basic)

- ☒ Access control enforced server-side
- ☒ Sensitive data minimised & protected
- ☒ Inputs validated against the contract
- ☒ Secrets kept out of code & logs

You cannot test everything, so test by risk

Rank each risk, then match test depth to its rank. Testing is a budget; spend it where failure hurts most.

$$\text{Likelihood} \times \text{Impact} = \text{Risk score} > \text{Test priority}$$



HIGH RISK

Deepest testing

Automation, edge cases, negative & security tests, load where relevant.



MEDIUM RISK

Standard coverage

Core functional cases plus the main boundary and error paths.



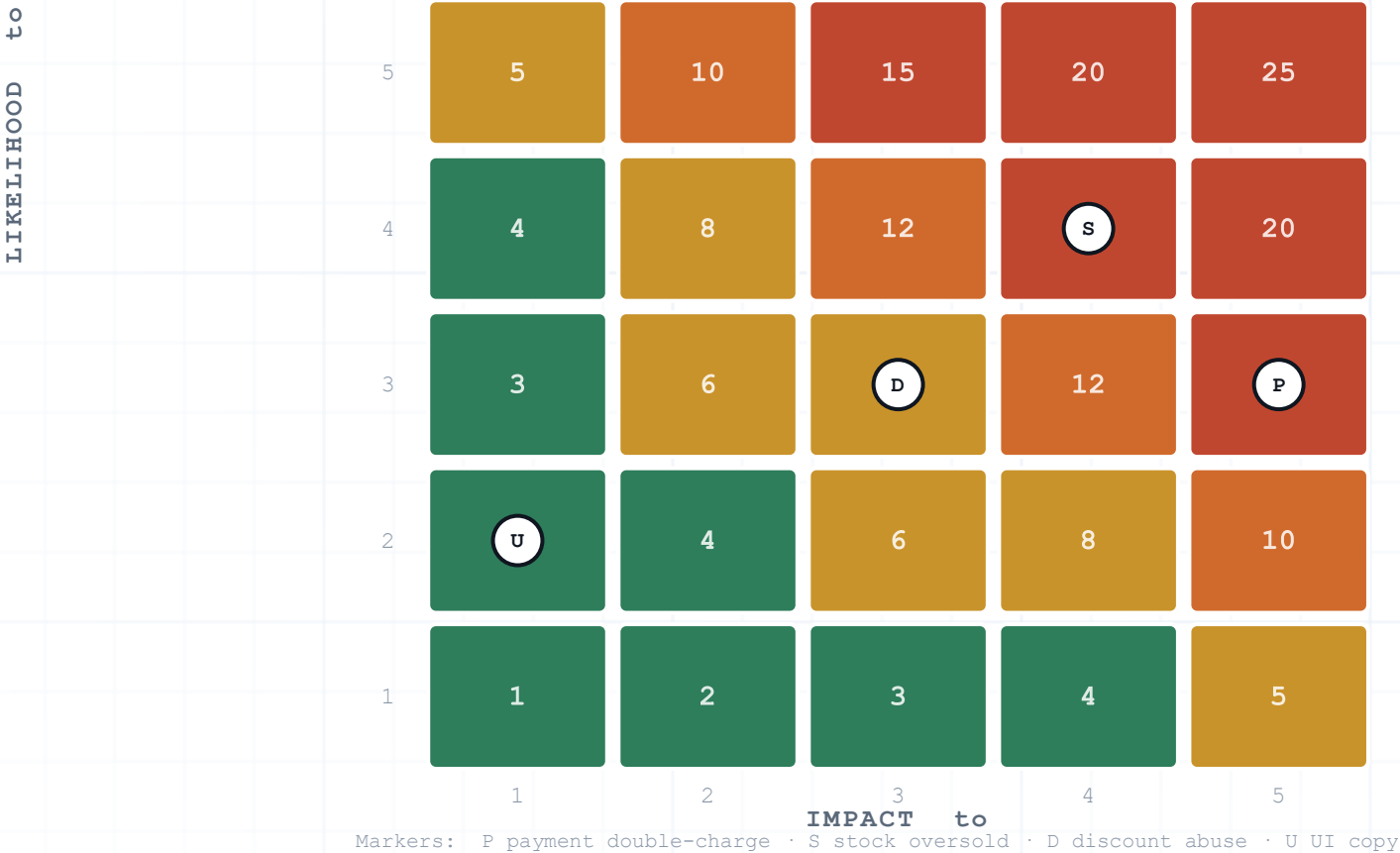
LOW RISK

Light touch

Smoke / sanity checks only, confirm it works, then move on.

Plot every risk on the 5 × 5 matrix

Likelihood × impact places each risk in a zone. The zone sets how hard you test it.



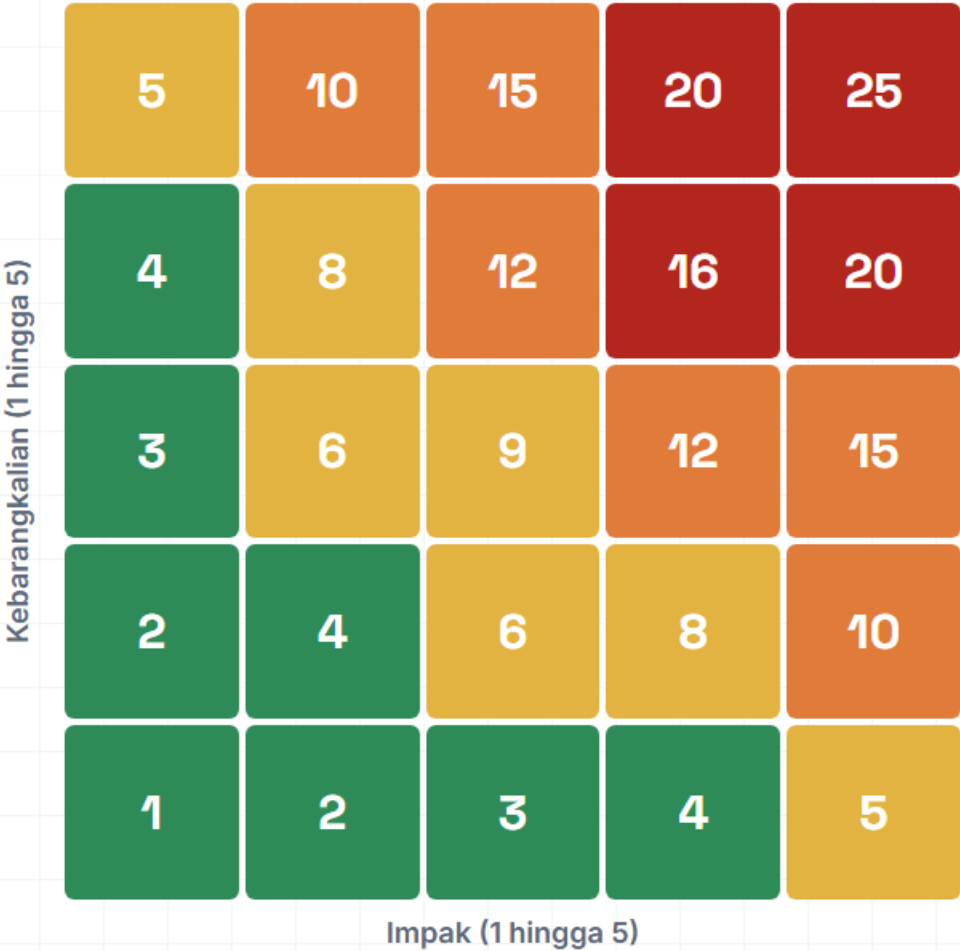
ZONE to TEST RESPONSE

- Critical (15-25)**
Test hardest: automation, security, load
- High (10-14)**
Thorough functional + negative + boundary
- Medium (5-9)**
Standard functional coverage
- Low (1-4)**
Smoke / sanity only

Matriks risiko 5 x 5: uji mengikut risiko

5 x 5 risk matrix: test by risk

Skor = Kebarangkalian x Impak. Merah dan jingga diuji dahulu dan paling mendalam. Daftar dalam templat PPrISA07.



ID	Risiko (contoh ShopFast)	K	I	Skor
R1	Order boleh dilihat tanpa semakan pemilik (IDOR)	4	5	20
R2	Diskaun salah kira semasa jualan perayaan	4	4	16
R3	Had kuantiti troli tidak dikuatkuasa	3	3	9
R4	Log merekod emel dan telefon (PDPA)	3	4	12
R5	Masa respons checkout perlahan	2	3	6

16 hingga 25 Kritikal

10 hingga 15 Tinggi

5 hingga 9 Sederhana

1 hingga 4 Rendah

From risk to test priority, ShopFast

The same four risks from the matrix, turned into a concrete, defensible testing plan.

RISK	L	I	SCORE	ZONE	TEST RESPONSE
Payment double-charge on retry	3	5	15	Critical	Idempotency tests + API automation + edge cases
Stock oversold on concurrent buys	4	4	16	Critical	Concurrency / load tests + regression suite
Discount-code abuse (new change)	3	3	9	Medium	Boundary + negative test cases
Address-field validation gap	3	2	6	Medium	Standard functional cases
Checkout button mislabeled	2	1	2	Low	Smoke check only

Scores are illustrative for the ShopFast exercise, your team sets its own likelihood and impact.

Design Review and Risk

Finding defects before code runs: reviews, inspections, walkthroughs, static analysis and the checklists behind them

01 Purpose and intents of quality reviews

02 Cost escalation and defect amplification

03 Formal technical reviews

04 Review types and formality

05 Roles, preparation and outcomes

06 Fagan inspections and walkthroughs

07 Static analysis techniques

08 Design and code review checklists

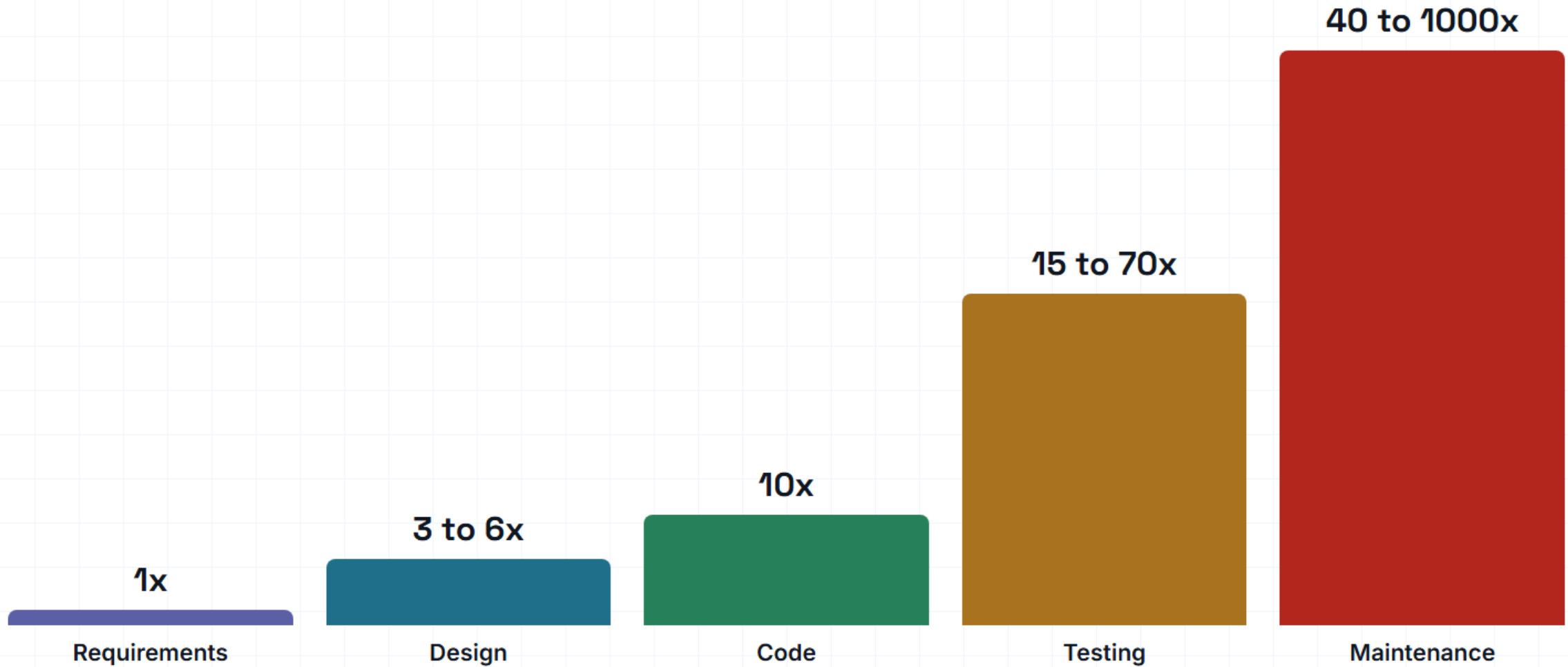
Why we review: purpose and intents

Reviews are the fundamental method of validating the quality of a product or a process

- **Applied at every phase:** Held during and at the end of each life cycle phase, not only just before release.
- **Point out improvements:** Show the author or team exactly where their work product needs to change.
- **Confirm what is good:** Record the parts where improvement is not needed, so effort is not wasted re-checking them.
- **Predictable quality:** Make technical work more uniform and predictable, and therefore more manageable.
- **Intent 1: defect removal:** Find errors in requirements, design or code while they are still cheap to fix.
- **Intent 2: progress:** Assess whether the work is really as complete as the schedule claims.
- **Intent 3: conformance:** Check consistency with standards, templates and earlier approved documents.

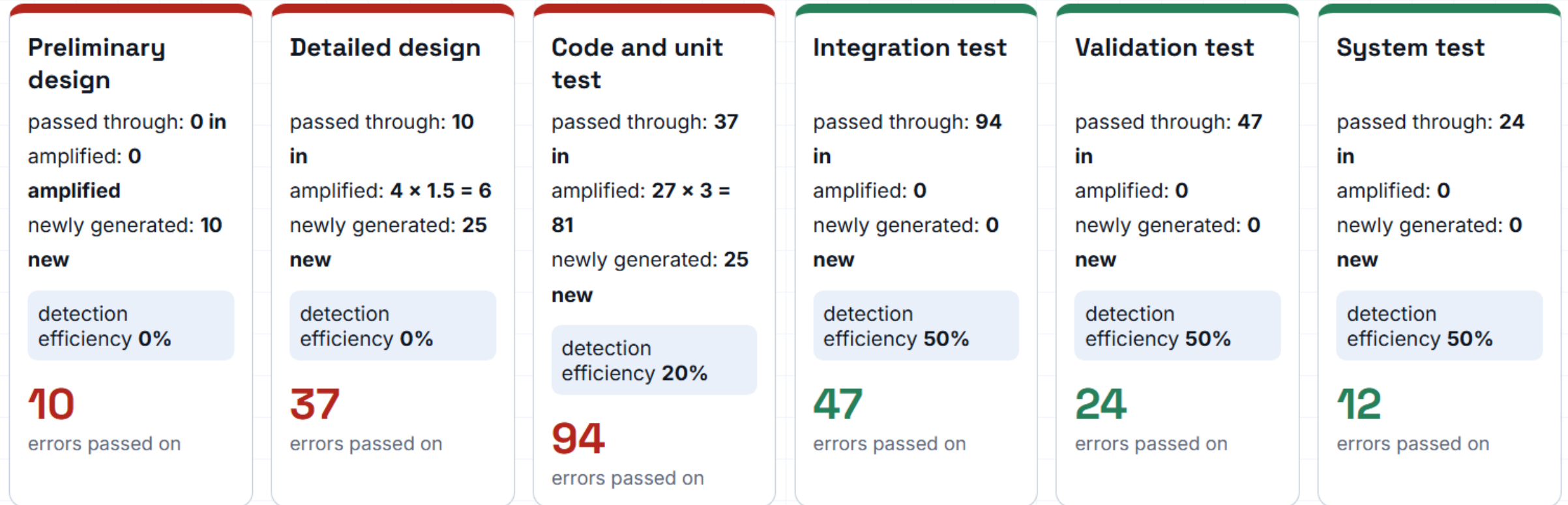
Relative cost to fix a defect, by where it is found

A review at each stage removes defects while they are still cheap



Defect amplification and removal (no reviews)

Errors not removed early are passed on and multiplied by later work; testing alone leaves latent errors



Design and code phases: no reviews, so 0% to 20% of errors are removed while new ones are added and old ones amplified.

Result: 12 latent errors reach the customer. Adding design and code reviews early cuts this number sharply.

Defect amplification, step by step

Pressman's example with no design or code reviews (figures as published)

Step	Errors arriving	New errors	Detection	Errors passed on
Preliminary design	0	10	0%	10
Detailed design	10: 6 passed through, 4 amplified x1.5 to give 6	25	0%	37
Code and unit test	37: 10 passed through, 27 amplified x3 to give 81	25	20% of 116	94
Integration testing	94	0	50%	47
Validation testing	47	0	50%	24
System testing	24	0	50%	12 latent errors

Formal Technical Review (FTR)

Formal Technical Review (FTR)

An SQA activity run by software engineering practitioners: a planned, stylised meeting whose sole purpose is to uncover quality problems in any representation of the software. It is a class of reviews: walkthroughs, inspections, round-robin reviews and more.

Uncover errors in function, logic or implementation

Verify that the work product meets its requirements

Assure it is represented according to predefined standards

Achieve uniform development and more manageable projects

Review types, from informal to formal

More formality costs more effort but finds more defects and produces evidence

less formal

more formal

Desk check

One person reviews alone and sends comments

Informal, no meeting

Peer review

Small group, prepared, moderator and recorder

Semi-formal

Walkthrough

Author leads the group through the product

Semi-formal, little preparation

Technical review

Team evaluates against requirements and standards

Formal, documented outcome

Inspection

Checklist-driven, defined roles, metrics collected

Most formal (Fagan)

Comparing review types by formality

Type	Who	Preparation	Records	Typical use
Desk inspection	One reviewer working alone	Individual reading	Comments sent to author, handled informally	Minor documents, small changes
Peer review	SQE, moderator, recorder, author(s), critical reviewers	Required from all; unprepared means ineffective	Issues documented on action forms (AR)	Plans, designs, important code
Walkthrough	Author or lead takes a group through the product	Mostly none in the Raytheon usage	Issues documented on CAR forms	Requirements traceability review
Technical review (FTR)	3 to 5 people: leader, reviewers, recorder, producer	Up to about 2 hours each	Summary report and issues list, sign-off	Phase-end specification or design
Inspection (Fagan)	Trained moderator, author, reader, inspectors	Individual study against a checklist	Defect log, metrics, verified follow-up	Completed design, completed code

The review meeting: constraints and roles

3 to 5 people

Typically three to five participants, which keeps the discussion focused.

Preparation up to 2 hours

Everyone prepares in advance, but for no more than about two hours each.

Meeting under 2 hours

Keep the meeting itself under two hours so attention stays high.

Producer

The person who developed the work product. Explains and answers, does not defend.

Review leader

Appointed by the project leader; checks readiness, copies the material and sends it to 2 or 3 reviewers.

Reviewers

Spend 1 to 2 hours studying the product, making notes and becoming familiar with it.

Recorder

Writes down every important issue raised, so nothing depends on memory.

Reviewer's preparation

- ☐ Understand the context: which requirement, phase and standard the material answers
- ☐ Skim all the material first to learn where information is and how it is formatted
- ☐ Read the product carefully and annotate your copy
- ☐ Write comments as questions, for example: What happens when quantity is 0?
- ☐ Avoid issues of style; focus on correctness, completeness and conformance
- ☐ Use the checklist for this product type and mark the items you verified
- ☐ Tell the review leader early if you cannot prepare

An unprepared participant makes the review ineffective. Reschedule rather than proceed (Tseytlin).

Guidelines for a good FTR

- **Product, not producer:** Criticise the work product, never the person. The tone decides whether people bring work for review early.
- **Keep the agenda:** Set an agenda and maintain it; limit debate and rebuttal.
- **Find, do not fix:** Name the problem areas, but do not try to solve every problem in the meeting.
- **Written notes:** Record issues in writing as they are raised.
- **Small and prepared:** Limit the number of participants and insist on advance preparation.
- **Checklists:** Develop a checklist for each type of product likely to be reviewed.
- **Plan, train, learn:** Schedule time for FTRs, train every reviewer, and review your earlier reviews.

Review outcomes and records

Accept

The product is accepted without further modification.

Reject

Severe errors found. Correct them, then hold another review.

Accept provisionally

Minor corrections are needed, but no further review is required.

Sign-off

Every attendee signs to show participation and agreement with the review team's findings.

Review summary report

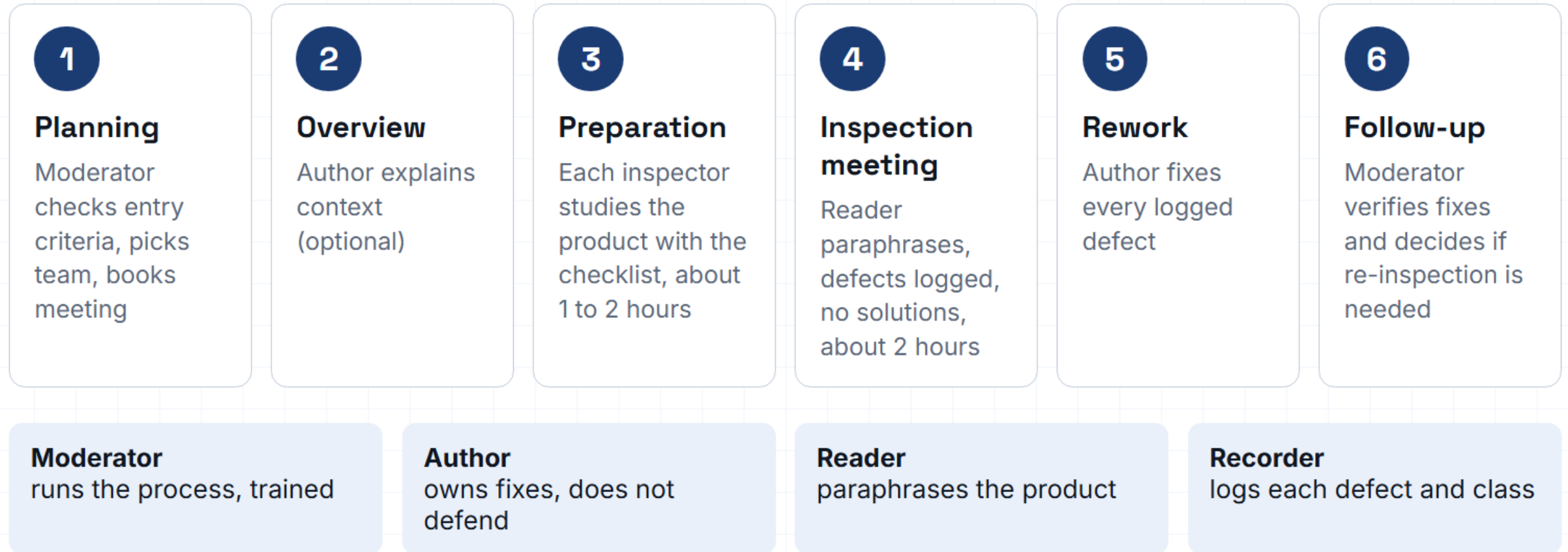
What was reviewed, who reviewed it, and what the findings and conclusions were.

Review issues list

Identifies problem areas and serves as the action checklist the producer works through.

Fagan inspection process

Introduced at IBM in 1972: checklist-driven, with defined roles and measured results



Reported effectiveness: finds about 70 to 80 percent of errors, most before unit testing. Pace about 100 statements per hour.

Structured walkthroughs and egoless programming

How a walkthrough runs

- Reviewee plus 3 to 5 reviewers; customers and users included, managers not
- Goal is to discover and note problems, not to resolve them in the session
- Major problems only; a follow-up meeting or memo reports actions taken
- A trained moderator keeps the atmosphere positive and non-threatening
- Never used as a vehicle for employee evaluation

Benefits reported

- Errors caught at the earliest possible time, greatly improving quality
- Better project communication and better project control
- Software easier to maintain; less disruption when staff are on leave
- New staff integrate faster and programmer expertise grows
- Emphasis shifts from private contemplation to clear communication
- Morale improves through involvement and social interaction

Egoless programming (Weinberg): the product belongs to the team, so finding its faults is help, not attack.

Static analysis techniques

Examining the program without executing it: tool checks plus human reviews

Technique	What it checks	ShopFast illustration
Syntax checks	Error-prone constructs and violations of coding standards	A linter flags <code>==</code> where <code>===</code> is required in <code>app.js</code>
Program structure checks	Builds graphs of the code to find structural flaws such as unreachable code	Statements after a <code>return</code> in the pricing function never run
Module interface checks	Inconsistent data declarations and improper links between modules	UI sends quantity as text while the API expects an integer
Event sequence checking	Compares event sequences in the code with the legal sequences in the spec	Code path lets ship act on a NEW order; spec allows ship only from PACKED
Symbolic execution	Runs code with symbols instead of values to derive results and path conditions	Total derived as $S - D + 8$ on the path where S is below 200
Formal verification	Theorem proving shows code is equivalent to a formal specification	Prove the total can never be negative for any cart
Human reviews	Checklist-driven inspections and walkthroughs	Design walkthrough of the checkout use case

Checklist: preliminary design review

- ☐ Are the software requirements reflected in the software architecture?
- ☐ Is the data structure consistent with the software requirements?
- ☐ Is effective modularity achieved? Are modules functionally independent?
- ☐ Has maintainability been considered?
- ☐ Is the program architecture factored?
- ☐ ShopFast tailoring: does each rule (quantity, promo codes, shipping, order lifecycle) map to one named module?
- ☐ Are interfaces defined for modules and for external system elements?

Adapted from Pressman. Tailor each generic checklist with items drawn from your own past defects.

Checklist: design walkthrough

- ☐ Does the algorithm accomplish the desired function?
- ☐ Is the algorithm logically correct?
- ☐ Is the interface consistent with the architectural design?
- ☐ Is the logical complexity reasonable?
- ☐ Have error handling and antibugging (defensive checks) been specified?
- ☐ Is local data structure properly defined?
- ☐ Are structured programming constructs used throughout?
- ☐ Is the design detail amenable to the implementation language?
- ☐ Which operating system or language dependent features are used, and are they justified?
- ☐ Is compound or inverse logic used? Has maintainability been considered?

Checklist: coding review

- ☐ Is the design properly translated into code? (Have the procedural design available at the review)
- ☐ Are there misspellings or typos?
- ☐ Have language conventions been used properly?
- ☐ Does the code comply with coding standards for style, comments and module prologue (header)?
- ☐ Are any comments incorrect or ambiguous?
- ☐ Are data typing and declarations proper?
- ☐ Are physical constraints correct (sizes, ranges, units, limits)?
- ☐ Have the design walkthrough checklist items been reapplied where required?
- ☐ ShopFast tailoring: do returned error codes match the API contract exactly?

Compilers and linters can automate many mechanical items, so reviewers can spend their time on logic and intent.

Your guided run on ShopFast

Three timeboxed steps. You leave with three artifacts that feed straight into Module 4.

1

12 min

Study the design

Open the ShopFast architecture diagram and the [OpenAPI](#) spec. Trace one checkout request end-to-end and note every trust boundary.

Architecture diagram · [Swagger UI](#)

2

13 min

Run the review

Walk the checklist, testability, error handling, security. Log each finding; flag anything untestable or unsafe by design.

Design Review Checklist · OWASP 2025

3

15 min

Build the risk plan

List what could go wrong, score likelihood × impact, plot the 5 × 5, and set a test priority per risk to your test strategy.

Risk matrix (Sheet) · Risk register



Carries forward: your risk ranking decides which test cases Module 4 writes first. Nothing here is throwaway.

What you walk away with

Three artifacts, added to the ShopFast quality trail, and mapped to a KRISA 2.0 deliverable.



Design Review Checklist

Completed, with every finding logged and rated.



Risk Register

Each risk scored by likelihood × impact and zoned.



Risk-Based Test Strategy

Test priorities and coverage plan, driven by risk.

KRISA 2.0

Beta 2026 · JDN

Chapter 4 · Design to System Design Specification (SDS)

Your architecture, API and data-model review evidence supports the SDS, the design deliverable an agency signs off before build.

Sample artifacts, templates & references

Grab-and-go examples in the format your deliverables should take, plus the standards behind them.

SAMPLE ROWS (copy the format)

```
// risk register
R-07 | Payment double-charge | L3×I5 = 15
      | Critical | Idempotency + API automation
```

```
// review finding
DR-02 | /checkout has no 409 defined | Major
      | Add stock-conflict response to spec
```

STANDARDS & GUIDELINES

-  **OWASP Top 10 : 2025**
owasp.org/Top10, current edition
-  **ISO/IEC 25010**
Product quality model (security, reliability...)
-  **ISO/IEC/IEEE 42010**
Architecture description
-  **KRISA 2.0 · Ch.4 Design**
System Design Spec (SDS) · JDN

LAB TOOLS

Swagger / OpenAPI

[draw.io](#)

Google Sheets

[Jira](#)

OWASP [Threat Dragon](#)

KRISA 2.0 is a Beta (2026); deliverable numbering may change, confirm the version with your agency.

Abbreviations used in this module (1 of 5)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
AI	Artificial Intelligence	Software that can answer questions and write text or code by learning patterns from large amounts of data.
API	Application Programming Interface	A doorway that lets one program ask another program for data or an action, without using a screen.
APM	Application Performance Monitoring	Tools that watch a live system and show how fast and healthy it is.
AR	Action Request	A form listing the issues found in a review that the author must fix.
BRS	Business Requirements Specification	The document that states what the business needs, written in business language. KRISA deliverable D02.
BVA	Boundary Value Analysis	Testing at the edges of the allowed values, such as 0, 1, 10 and 11, because mistakes often hide there.
CAR	Corrective Action Request	A form raised when a review or audit finds something that must be corrected.
CI	Continuous Integration, or Configuration Item	Two meanings. In DevOps, every code change is built and tested automatically. In configuration management, it is any controlled item such as a document or code file.
CI/CD	Continuous Integration and Continuous Delivery	An automated pipeline that builds, tests and prepares every change for release.

Abbreviations used in this module (2 of 5)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
CSV	Comma-Separated Values	A simple table file that opens in Excel, with commas between the columns.
DB	Database	Where the system keeps its data, such as orders and customers.
DPP	Dokumen Permulaan Projek	The Project Initiation Document. PPrISA template PPrISA01.
DR	Design Review	A formal check of a design. In the lab, IDs such as DR-02 label design review findings.
EP	Equivalence Partitioning	Grouping inputs that behave the same way, then testing one value from each group.
FTR	Formal Technical Review	A planned meeting where colleagues check a piece of work for defects, with set roles and written records.
IBM	International Business Machines	A large technology company. Its research is often quoted on the cost of fixing defects late.
ICT	Information and Communication Technology	Computers, networks and software used to handle information.
ID	Identifier	A unique label, such as REQ-CHK-01, so that each item can be traced.

Abbreviations used in this module (3 of 5)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
IDOR	Insecure Direct Object Reference	A security hole where changing an ID in a request lets a user see someone else's data, such as another customer's order.
IEC	International Electrotechnical Commission	An international standards body that publishes many software standards together with ISO.
IEEE	Institute of Electrical and Electronics Engineers	A professional body that publishes software engineering standards such as IEEE 730 and IEEE 829.
ISO	International Organization for Standardization	Publishes international standards such as ISO 9001 and ISO/IEC 25010.
JDN	Jabatan Digital Negara	The Malaysian government department that publishes KRISA and PPrISA.
KRISA	Kejuruteraan Sistem Aplikasi Sektor Awam	The Malaysian public sector guide for engineering application systems. KRISAv2 is the 2026 beta edition.
MIT	Massachusetts Institute of Technology	A university in the United States. Source of the Leveson course notes.
OWASP	Open Worldwide Application Security Project	A community that publishes free web security guides such as the OWASP Top 10.

Abbreviations used in this module (4 of 5)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
PAT	Provisional Acceptance Test	The KRISA acceptance test done before provisional acceptance of the system, including performance checks.
PDF	Portable Document Format	A document file that looks the same on every computer.
PDPA	Personal Data Protection Act 2010	The Malaysian law on how personal data must be collected, used and protected.
PII	Personally Identifiable Information	Data that can identify a person, such as a name, IC number or card number.
PPP	Pelan Pengurusan Projek	The Project Management Plan. PPrISA template PPrISA02.
PPrISA	Panduan Pengurusan Projek ICT Sektor Awam	The Malaysian public sector guide for managing ICT projects. Codes such as PPrISA05 are its templates.
PS	PowerShell prompt	When a slide shows PS>, type that command in Windows PowerShell.
QA	Quality Assurance	Preventing defects by setting up good processes and checking that people follow them.

Abbreviations used in this module (5 of 5)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
RTM	Requirements Traceability Matrix	A table linking each requirement to its design, tests and defects, so nothing is missed.
SDS	Spesifikasi Reka Bentuk Sistem	The System Design Specification. KRISA deliverable D04.
SQA	Software Quality Assurance	All the planned work that gives confidence the software meets its quality requirements.
SQE	Software Quality Engineer	The person who runs quality checks and audits and witnesses formal tests.
SRS	Spesifikasi Keperluan Sistem	The System (or Software) Requirements Specification: what the system must do. KRISA deliverable D03.
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege	Six kinds of security threat to look for in a design review.
UAT	User Acceptance Test	Users try the system to confirm it meets their needs before they accept it.
UI	User Interface	The screens, buttons and forms that people use.

References and templates for this module

Every line opens the official JDN document at the exact page. Templates download as Word or PDF.

WHERE IT IS IN THE GUIDES

- Bab 4, 4.5 Reka Bentuk Seni Bina
page 4 · KRISAv2 Bab 4 Fasa Reka Bentuk
- Bab 4, Jadual 4.4 Jenis seni bina (DevOps, CI/CD)
page 10 · KRISAv2 Bab 4 Fasa Reka Bentuk
- Bab 4, 4.12 Spesifikasi Integrasi
page 111 · KRISAv2 Bab 4 Fasa Reka Bentuk
- Bab 1, 1.6.7 Faktor Keselamatan ICT, Jadual 1.5
page 24 · KRISAv2 Bab 1 Perancangan
- Bab 5, 5.6.8 Amalan Pengaturcaraan Selamat
page 30 · KRISAv2 Bab 5 Fasa Pembangunan
- PPrISA 2.0, 4.2.3 a) Pengurusan Risiko
page 71 · PPrISA 2.0 Versi Beta Feb 2025

TEMPLATES TO USE

- D04
Spesifikasi Reka Bentuk Sistem (SDS)

WORD

PDF
- D08
Spesifikasi Integrasi Data

WORD

PDF
- PPrISA07
Pelan Pengurusan Risiko

WORD

PDF
- PPrISA12
Borang Pelaporan Risiko

WORD

PDF
- PPrISA13
Log Penyelesaian Risiko

WORD

PDF

Prevent early. Secure by design. Test by risk.



Prevention beats detection

The cheapest defect is one caught while the design is still on paper.



Security is a design decision

Review against OWASP Top 10 : 2025 and threat-model with STRIDE.



Test by risk

Likelihood × impact tells you exactly where to aim the deepest testing.

NEXT to **Module 4 · Test Design & Test Management, your risk ranking decides which cases to write first.**

Al-Abrar bin Abdul Wahid

SQA Trainer · Aligned to KRISA 2.0 (JDN)



abrar.abdulwahid@gmail.com



+60 13-283 5415



LinkedIn

M03 · W03 · 20 MINUTES · Hari 1, 15:05 hingga 15:25

Hands-On SQA-AI Assistant Workshop

AI design review and 5 x 5 risk register

Review the ShopFast architecture and OpenAPI contract with the AI, challenge one finding for evidence, then build a scored risk register in PPrISA07 format.

TOOL

opencode 1.18 in the lab folder · model: Big Pickle via OpenCode Zen (free, no API key) · synthetic ShopFast data only

[Step-by-step guide \(web\)](#)[Workshop slides \(W03\)](#)[Setup guide W00](#)[opencode docs](#)

AI design review and 5 x 5 risk register

STEPS

- 1 Run /design-review on the order endpoint
- 2 Challenge one finding: ask for the exact line as evidence
- 3 Run /risk-register and approve the CSV
- 4 Sort by score and agree the top three test priorities

OUTPUT TO SAVE

outputs/m03-risk-register.csv

TYPE THIS (opencode> in opencode, PS> in PowerShell)

```
opencode> /design-review GET /api/orders/{orderId}

opencode> Petik baris tepat dalam shopfast/docs/openapi.yaml yang
membuktikan penemuan anda tentang GET /api/orders/{orderId}.

opencode> /risk-register checkout ShopFast berdasarkan penemuan semakan
reka bentuk di atas
```

HUMAN CHECK BEFORE YOU ACCEPT IT

- Findings quote the exact path or line in openapi.yaml or architecture.md
- Scores use 1 to 5 scales and Skor = K x I
- Each top risk has a test priority, not only a mitigation

Open step-by-step guide

Module hub

Pembantu AI dalam SQA: AI mendraf, manusia bertanggungjawab

AI assistant in SQA: AI drafts, humans remain accountable

Aliran kerja setiap bengkel Hands-On SQA-AI Assistant Workshop menggunakan opencode

1

Konteks

@SRS, @templat,
AGENTS.md

2

Prompt

arahan /review-srs

3

Draf AI

opencode + model

4

Semak manusia

banding dengan templat
KRISA/PPrISA

5

Simpan bukti

outputs/ + git commit

Guna AI untuk

Draf pelan dan laporan, cadang kes ujian
EP/BVA, jana skrip ujian, ringkaskan log, cari
jurang RTM.

Jangan

Tampal data sebenar agensi, terima output
tanpa semak, biar AI reka ID keperluan, kongsi
sesi awam.

Bukti audit

Simpan prompt, output dan semakan dalam git
supaya jejak audit lengkap (Modul 7).